Lab Exercise 15- Creating ConfigMaps in Kubernetes

In this lab exercise, you will learn how to create ConfigMaps in Kubernetes using different methods: from literal values, from a file, from an environment file, and from multiple files. ConfigMaps allow you to decouple configuration artifacts from image content to keep containerized applications portable.

Step 1: Set Up Kubernetes Cluster

Ensure you have access to a Kubernetes cluster. You can use a local setup with Minikube, kind, or use a cloud-based Kubernetes service.

Step 2: Create a ConfigMap from Literal Values

Create a ConfigMap named example-configmap with literal values:

```
kubectl create configmap example-configmap --from-literal=key1=value1 --from-literal=key2=value2
```

```
C:\Users\shwet>kubectl create configmap example-configmap --from-literal=key1=value1 --from-literal=key2=value2
configmap/example-configmap created
```

Verify the ConfigMap:

```
kubectl get configmap example-configmap -o yaml
```

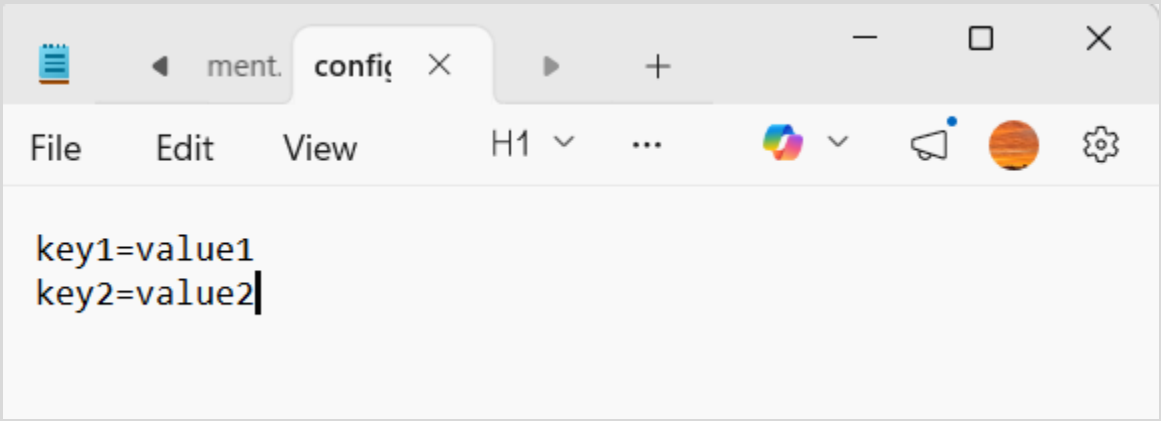
```
C:\Users\shwet>kubectl get configmap example-configmap -o yaml
apiVersion: v1
data:
  key1: value1
  key2: value2
kind: ConfigMap
metadata:
  creationTimestamp: "2026-04-07T17:52:14Z"
  name: example-configmap
  namespace: default
  resourceVersion: "24446"
  uid: 26fc7bcf-774b-429a-8faa-086f29fc71af
```

You should see the example-configmap with the keys and values specified.

Step 3: Create a ConfigMap from a File

Create a file named **config.txt** with the following content:

```
key1=value1
key2=value2
```



Create a ConfigMap named file-configmap from the file:

```
kubectl create configmap file-configmap --from-file=config.txt
```

```
C:\Users\shwet>kubectl create configmap file-configmap --from-file=config.txt
configmap/file-configmap created
```

Verify the ConfigMap:

kubectl get configmap file-configmap -o yaml

```
C:\Users\shwet>kubectl get configmap file-configmap -o yaml
apiVersion: v1
data:
  config.txt: "key1=value1\r\nkey2=value2"
kind: ConfigMap
metadata:
  creationTimestamp: "2026-04-07T17:53:51Z"
  name: file-configmap
  namespace: default
  resourceVersion: "24575"
  uid: 2bb1a045-128f-4f89-8078-cc3b31c9dd6f
```

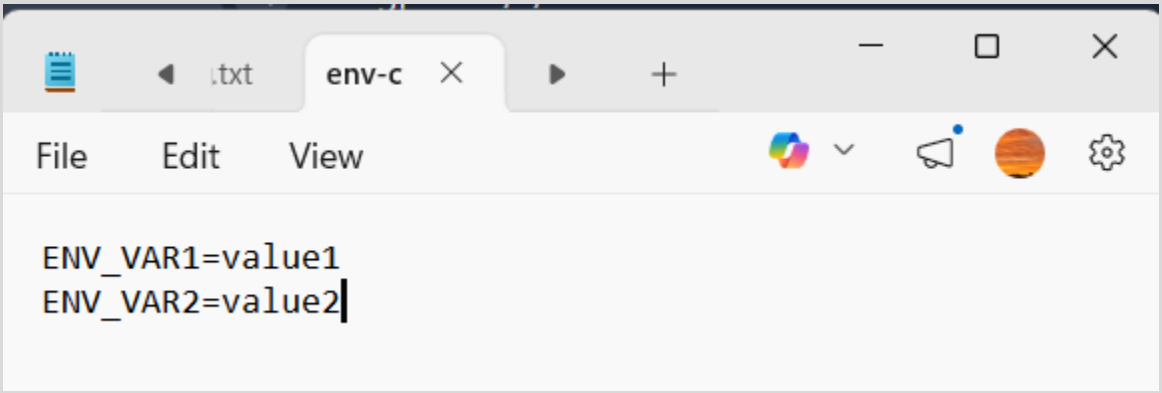
You should see the file-configmap with the content of config.txt.

Step 4: Create a ConfigMap from an Environment File

Create an environment file named **env-config.env** with the following content:

ENV_VAR1=value1

ENV_VAR2=value2



Create a ConfigMap named env-configmap from the environment file:

```
kubectl create configmap env-configmap --from-env-file=env-config.env
```

```
C:\Users\shwet>kubectl create configmap env-configmap --from-env-file=env-config.env
configmap/env-configmap created
```

Verify the ConfigMap:

```
kubectl get configmap env-configmap -o yaml
```

```
C:\Users\shwet>kubectl get configmap env-configmap -o yaml
apiVersion: v1
data:
  ENV_VAR1: value1
  ENV_VAR2: value2
kind: ConfigMap
metadata:
  creationTimestamp: "2026-04-07T17:56:12Z"
  name: env-configmap
  namespace: default
  resourceVersion: "24760"
  uid: 79352609-bd19-4603-b2ec-0d3522c6e0ae
```

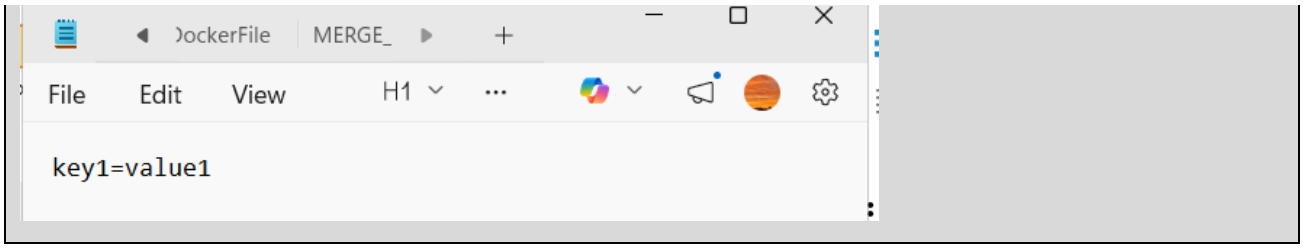
You should see the env-configmap with the environment variables specified in the file.

Step 5: Create a ConfigMap from Multiple Files

Create multiple files:

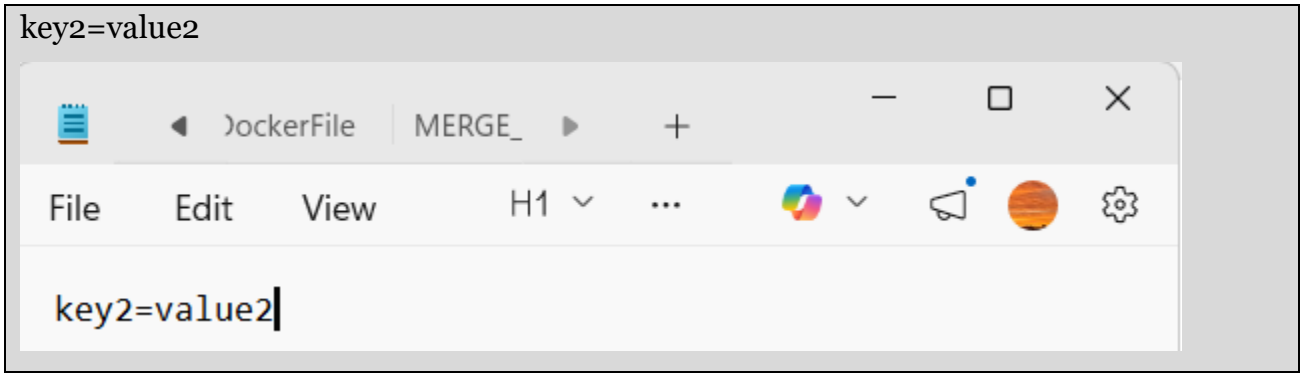
config1.txt with content:

```
key1=value1
```



config2.txt with content:

```
key2=value2
```



Create a ConfigMap named multi-file-configmap from multiple files:

```
kubectl create configmap multi-file-configmap --from-file=config1.txt --from-file=config2.txt
```

```
C:\Users\shwet>kubectl create configmap multi-file-configmap --from-file=config1.txt --from-file=config2.txt
configmap/multi-file-configmap created
```

Verify the ConfigMap:

```
kubectl get configmap multi-file-configmap -o yaml
```

```
C:\Users\shwet>kubectl get configmap multi-file-configmap -o yaml
apiVersion: v1
data:
  config1.txt: key1=value1
  config2.txt: key2=value2
kind: ConfigMap
metadata:
  creationTimestamp: "2026-04-07T17:57:37Z"
  name: multi-file-configmap
  namespace: default
  resourceVersion: "24877"
  uid: f13f26bc-8e0b-4099-b618-0387fd13aa7e
```

You should see the multi-file-configmap with the content of both config1.txt and config2.txt.

Step 6: Use ConfigMaps in a Pod

Create a file named **pod-configmap.yaml** with the following content:

```
apiVersion: v1
kind: Pod
metadata:
  name: configmap-pod
spec:
  containers:
  - name: nginx
    image: nginx
    envFrom:
    - configMapRef:
        name: env-configmap
```

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: configmap-pod
spec:
  containers:
  - name: nginx
    image: nginx
    envFrom:
    - configMapRef:
        name: env-configmap
```

In this manifest:

- The Pod reads environment variables from the env-configmap.
- The Pod mounts the file-configmap at /etc/config.

Apply the manifest to create the Pod:

```
kubectl apply -f pod-configmap.yaml
```

```
C:\Users\shwet>kubectl apply -f pod-configmap.yaml
pod/configmap-pod created
```

Check the status of the Pod:

```
kubectl get pods
```

```
C:\Users\shwet>kubectl get pods
NAME            READY   STATUS    RESTARTS      AGE
configmap-pod   1/1     Running   0             20s
my-pod          1/1     Running   3 (12m ago)   56d
```

View the Pod logs to see the environment variables and file content:

kubectl exec -it configmap-pod – printenv

```
C:\Users\shwet>kubectl exec -it configmap-pod -- printenv
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
HOSTNAME=configmap-pod
TERM=xterm
ENV_VAR1=value1
ENV_VAR2=value2
KUBERNETES_SERVICE_PORT=443
NODEPORT_SERVICE_SERVICE_HOST=10.98.155.180
KUBERNETES_SERVICE_HOST=10.96.0.1
KUBERNETES_PORT_443_TCP_ADDR=10.96.0.1
NODEPORT_SERVICE_PORT_80_TCP=tcp://10.98.155.180:80
NODEPORT_SERVICE_PORT_80_TCP_PORT=80
KUBERNETES_SERVICE_PORT_HTTPS=443
KUBERNETES_PORT=tcp://10.96.0.1:443
KUBERNETES_PORT_443_TCP_PROTO=tcp
KUBERNETES_PORT_443_TCP_PORT=443
NODEPORT_SERVICE_PORT=tcp://10.98.155.180:80
KUBERNETES_PORT_443_TCP=tcp://10.96.0.1:443
NODEPORT_SERVICE_SERVICE_PORT=80
NODEPORT_SERVICE_PORT_80_TCP_PROTO=tcp
NODEPORT_SERVICE_PORT_80_TCP_ADDR=10.98.155.180
NGINX_VERSION=1.29.7
NJS_VERSION=0.9.6
NJS_RELEASE=1~trixie
ACME_VERSION=0.3.1
PKG_RELEASE=1~trixie
DYNPKG_RELEASE=1~trixie
HOME=/root
```

You should see the environment variables and the content of the file-configmap.

Step 7: Clean Up

After completing the exercise, clean up the resources created:

kubectl delete pod configmap-pod

kubectl delete configmap example-configmap

kubectl delete configmap file-configmap

kubectl delete configmap env-configmap

kubectl delete configmap env-configmap

rm config.txt env-config.env config1.txt config2.txt

```
C:\Users\shwet>kubectl delete pod configmap-pod
pod "configmap-pod" deleted

C:\Users\shwet>kubectl delete configmap example-configmap
configmap "example-configmap" deleted

C:\Users\shwet>kubectl delete configmap file-configmap
configmap "file-configmap" deleted

C:\Users\shwet>kubectl delete configmap env-configmap
configmap "env-configmap" deleted
```

```
C:\Users\shwet>del config.txt env-config.env config1.txt config2.txt
```